BITÁCORA DE DESARROLLO CON IA

PROYECTO CINEBUSCA

Registro técnico de prompts, integración de herramientas IA y desarrollo de la aplicación de búsqueda de películas

REPOSITORIO OFICIAL	HERRAMIENTAS IA	API UTILIZADA
rub803030/cinebusca	Big Pickle (OpenCode)	TMDB · The Movie Database

01. REGISTRO HISTÓRICO DE PROMPTS Y MODIFICACIONES

PROMPT 01

Big Pickle

> *"Necesito una aplicación web pequeña que consuma una API externa para una tarea. ¿Qué API gratuita me recomiendas para un buscador de películas y cómo estructuro el proyecto con Node y Express?"*

Resultado / Implementación: Se definió TMDB por su plan gratuito para uso educativo y la estructura del proyecto: *server.js*, *lib/*, *public/*, *test/* y *docs/*. Se descartó React para poder explicar cada línea del código.

PROMPT 02

Big Pickle

> *"Genera el backend en Express que consulte TMDB sin exponer la clave en el navegador, con .env y .gitignore, y que devuelva errores claros si falta la clave o si TMDB falla."*

Resultado / Implementación: Se creó un cliente con autenticación v3 y v4, timeout de 8 segundos y errores con código propio; las rutas */api/movies/**, la caché de 5 minutos y un manejador central de errores. La clave solo vive en el servidor.

PROMPT 03

Big Pickle

> *"Haz el frontend en HTML, CSS y JavaScript puro: buscador, cuadrícula de pósters responsive y un modal con el detalle de la película. Sin frameworks y sin claves en el cliente."*

Resultado / Implementación: Interfaz con búsqueda en vivo (debounce de 500 ms), esqueletos de carga y un *<dialog>* nativo para el detalle. No funcionó a la primera: el botón "Cargar más" no se ocultaba (ver sección 02).

PROMPT 04

Big Pickle

> *"¿Cómo manejo en el frontend los errores 429, 401 y los timeouts para que el usuario vea mensajes distintos y pueda reintentar?"*

Resultado / Implementación: Un mapa de códigos a mensajes (*RATE_LIMIT*, *AUTH*, *TIMEOUT*, *OFFLINE*…), una clase *ApiError* propia y un botón que repite la última petición. Se agregó el tiempo de espera de *Retry-After*.

01. REGISTRO HISTÓRICO DE PROMPTS (CONTINUACIÓN)

PROMPT 05 — Big Pickle

> *"Escríbeme pruebas con node:test para el cliente de TMDB y para los endpoints, incluyendo el caso en que no hay clave configurada."*

Resultado / Implementación: 19 pruebas que cubren URL y parámetros, token v4, error 429 con *Retry-After*, 401, timeout, fallo de red, JSON inválido, contrato de datos, caché y validaciones. Una prueba de caché era no determinista y se corrigió.

PROMPT 06 — GitHub / Git

> *"Revisa el código y dime qué problemas de seguridad o accesibilidad tiene antes de subirlo a GitHub."*

Resultado / Implementación: Se añadieron cabeceras de seguridad y una CSP que solo permite imágenes de *image.tmdb.org*; se confirmó el uso de *textContent* en lugar de *innerHTML*, más *aria-live*, enlace para saltar al contenido y foco visible. El proyecto se publicó en *github.com/rub803030/cinebusca* con historial de commits.

02. ERROR DE LA IA DETECTADO Y CORRECCIÓN

El botón «Cargar más» no se ocultaba

El JavaScript asignaba correctamente *hidden = true*, pero la regla CSS *.btn { display: inline-flex }* anulaba el *display: none* que el navegador aplica al atributo *hidden*. El botón seguía visible y al pulsarlo se pedía una página inexistente.

Se detectó probando la interfaz y se corrigió con una regla explícita:

```
[hidden] {
  display: none !important;
}
```

También se corrigió el plural: la interfaz mostraba «1 resultados» y «1 películas cargadas».

03. RESUMEN DEL FLUJO DE TRABAJO Y APRENDIZAJES

- **Probar en el navegador, no solo leer el código:** los dos errores anteriores compilaban y pasaban las pruebas del backend, pero solo se veían al usar la aplicación.
- **Pedir los casos de error desde el principio:** al enumerar sin conexión, 429, 401, timeout y respuesta vacía, el resultado fue mucho más completo que con un «maneja errores» genérico.
- **Fijar restricciones explícitas:** «sin frameworks», «sin claves en el cliente» y «explícame cada archivo» evitaron propuestas que no pudiera defender en la demo.
- **Documentar y publicar:** el README, la bitácora y los commits en GitHub dejan trazabilidad del proceso, no solo del resultado final.